



Business Rules & Calculation Specification

Mathematical Formulations for Accounting, EMI, COGS, Stock & P&L

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Financial Specification	Accounting Specialists, Core Engineers	v1.4.0 Production	✓ Verified & Approved

Business Rules & Calculation Specifications

This document outlines all core business logic, calculation formulas, and state transition rules for StockWhisk.

1. Cash Flow & Accounting Rules

The financial engine operates on a strict double-entry ledger-like system tailored for retail operations.

- Golden Rule:** $\text{Expected Cash} = \text{Opening Balance} + \sum \text{Inflows} - \sum \text{Outflows}$
- Inflows:**
 - Sales payments (cash/digital received)
 - Due collections
 - Capital investments (Owner contributions)
- Outflows:**
 - Purchase payments to suppliers
 - Operating expenses
 - Sale refunds
 - Owner drawings
- COGS Rule:** Cost of Goods Sold (COGS) is snapshotted at the time of sale (`SaleItem.unit_cost`). It is NEVER deducted again from cash on sales. Profit is calculated as $\text{Sale Price} - \text{COGS}$.
- Account Transfer Rule:** Moving money between internal accounts (e.g., Cash → bKash → Bank) does NOT affect P&L. It only rebalances liquid accounts via offsetting LedgerEntries.
- Drawing Rule:** Owner withdrawals reduce equity/capital and liquid cash, NOT operating expenses. They do not affect the P&L statement.
- Immutability:** Ledger entries are append-only and immutable. Corrections require a reversing entry to maintain a strict audit trail.

2. Sales & POS Rules

- Visibility:** Zero-stock products are excluded from the POS grid (filter: `current_stock > 0`), unless `track_inventory=False` .

- **Cart Total Calculation:**

$\text{Cart total} = \sum(\text{item.price} \times \text{qty}) - \text{discount} + \text{delivery_charge} + \text{VAT}$

- **Sale Status:**

- **PAID** : $\text{paid} \geq \text{total}$
- **PARTIAL** : $0 < \text{paid} < \text{total}$
- **DUE** : $\text{paid} = 0$

- **Payments:** A single sale can accept multiple payment methods (Cash, bKash, Nagad, Card, Bank) simultaneously.
- **Quotation Mode:** Saves an invoice document without deducting stock or affecting accounting ledgers.

3. EMI Calculations

Equated Monthly Installment (EMI) sales follow these standard formulas:

- **Principal:** $\text{Total Invoice Amount} - \text{Down Payment}$
- **Interest:** $\text{Principal} \times (\text{Interest Rate \%} / 100)$
- **Total EMI Amount:** $\text{Principal} + \text{Interest}$
- **Monthly Installment:** $\text{Total EMI Amount} / \text{Number of Months}$

4. Purchase Rules

- **Cash Purchase:** Immediately reduces liquid funds (via `LedgerEntry`) and increases stock.
- **Credit Purchase:** No immediate cash deduction. Increases stock and increases Supplier Payable (`due_balance`).
- **Receiving PO:** Increments `Product.current_stock` via a stock movement function `apply_movement(PURCHASE_IN)` .
- **Serialization:** If a product is serialized, receiving a PO creates individual `ProductUnit` records for each item quantity.

5. Returns & Exchanges Rules

- **Return with Restock:** Stock is added back via `apply_movement(SALE_RETURN_IN)` .
- **Serialized Return:** The specific `ProductUnit` status reverts to `IN_STOCK` , `DEFECTIVE` , or `RETURNED_SUPPLIER` based on condition.
- **Refund:** Creates a negative `LedgerEntry` to deduct from the respective cash/bank account and adjusts total sales.
- **Exchange:** Swaps a defective unit (moves to `TESTING_PENDING`) for a new `IN_STOCK` unit. Rolls over the existing warranty to the new unit.

6. Due Collection Rules

- **Nature:** Due collection is NOT a new sale; it is debt settlement.
- **Accounting:** Increases liquid funds (positive `LedgerEntry`).
- **Customer:** Decreases `Customer.due_balance` .
- **Sale Record:** Updates the specific `Sale` paid amount and recalculates its status (e.g., from `PARTIAL` to `PAID`).

7. Settlement/Shift Closing Rules

Designed for daily register reconciliation.

- **Expected Cash Calculation:** $\text{Expected Cash} = \text{Opening Cash} + \text{Today's Cash Inflows} - \text{Today's Cash Outflows}$
- **Discrepancy:** $\text{Actual Counted Cash} - \text{Expected Cash}$ (Positive = Overage, Negative = Shortage).
- **Carry-over:** Closing cash carries over as the next day's opening cash automatically.
- **Backfill:** Missing days (days with no settlement) are auto-backfilled to maintain continuity.

8. Inventory Rules

- **Ledger:** Stock is tracked via an append-only `StockMovement` ledger.
- **Cache:** `Product.current_stock` is a derived/cached value updated strictly via triggers or movement signals.
- **Movement Types:** `PURCHASE_IN`, `SALE_OUT`, `SALE_RETURN_IN`, `ADJUSTMENT`, `DAMAGE_OUT`, `TRANSFER_IN`, `TRANSFER_OUT`.
- **Service Items:** If `track_inventory=False` (e.g., labor, digital goods), no stock deduction occurs on sale.

9. Warranty Rules

- **Calculation:** $\text{expiry_date} = \text{start_date} + \text{period_months}$
- **Status Indicators:**
 - `ACTIVE`: Current date is before expiry.
 - `EXPIRING_SOON`: Current date is within 30 days of expiry.
 - `EXPIRED`: Current date is past expiry.
- **Binding:** Warranties are bound to a specific `ProductUnit` (serial number) for serialized items, or the invoice/item for non-serialized.

10. Service Ticket Rules

- **State Machine:** `RECEIVED` → `DIAGNOSING` → `AWAITING_PARTS` → `IN_REPAIR` → `READY` → `DELIVERED` (or `CANCELLED`).
- **Billing Calculation:** $\text{Customer Bill} = \text{Labor (service_charge)} + \sum \text{Parts_Sell_Price} - \text{Discount}$
- **Inventory Integration:** Parts used with `from_stock=True` automatically deduct from shop inventory upon ticket completion.